

SILVER COMMODITY PRICE FORECASTING AGENT

Internship / Minor Project Report

A multi-model forecasting and market analytics platform for silver prices

ARIMA | Prophet | XGBoost | LSTM | Ensemble Methods

Interactive analytics dashboard and historical trading arena

Project	Commodity Price Forecasting Agent
Commodity	Silver
Test Period	2021-01-01 to 2025-12-31
Implementation	Python, Pandas, scikit-learn, XGBoost, TensorFlow, Prophet, Streamlit, Power BI / HTML analytics

Prepared from the implemented project pipeline and generated outputs

Abstract

This project develops an end-to-end silver commodity price forecasting and analytics platform. The implemented pipeline combines daily OHLCV data with macroeconomic indicators, futures-market variables, positioning and sentiment signals to construct a feature-rich analytical dataset. Four forecasting families - ARIMA baseline, Prophet with external regressors, XGBoost regression, and an LSTM neural network - are evaluated on an out-of-sample test period from 2021 through 2025. Multiple ensemble strategies are also produced, together with market-regime, volatility and value-at-risk style analytics. The project is completed with a Streamlit trading arena for historical simulation and an interactive market analytics dashboard exported as PDF/HTML.

Executive Summary

- The engineered/merged analytical dataset contains 4,435 rows and 101 columns, spanning 2009-01-01 to 2025-12-31 after cross-source alignment and rolling-feature warm-up.
- The final out-of-sample test set contains 1,304 observations from 2021-01-01 to 2025-12-31.
- XGBoost is the strongest evaluated model on the reported test set, with RMSE 0.2680, MAE 0.2046, MAPE 0.7345% and directional accuracy 85.11%.
- The application layer provides model selection, forecast visualisation, risk monitoring, historical trading simulation and crisis-scenario exploration.
- The platform is designed as an analytical prototype rather than a live trading system; forecasts are historical test outputs and the trading arena is a simulation.

Table of Contents

1. Introduction
2. Objectives and Scope
3. Data Sources and Dataset Engineering
4. System Architecture
5. Feature Engineering
6. Forecasting Methodology
7. Ensemble and Risk Analytics
8. Evaluation and Results
9. Dashboard and Trading Arena
10. Limitations and Responsible Interpretation
11. Conclusion and Future Scope

Appendix A - Project Structure and Reproduction

1. Introduction

Commodity price forecasting is a difficult time-series problem because market prices react to technical momentum, liquidity, macroeconomic conditions, investor positioning and rapidly changing expectations. Silver is particularly interesting because it behaves both as a precious metal and as an industrial commodity. The project therefore uses a multi-source modelling approach rather than relying only on historical closing prices.

The project brief requested a comprehensive forecasting system supported by several modelling families, advanced ensemble methods, interactive analytics and a gamified trading arena. The implementation delivered here focuses on a practical, reproducible prototype that demonstrates those capabilities without requiring live market APIs.

1.1 Problem Statement

Design an analytical platform that prepares a multi-source silver dataset, generates informative technical and economic features, trains several forecasting models, evaluates out-of-sample behaviour, produces risk and regime analytics, and exposes the outputs through an interactive user interface.

1.2 Why a Multi-Model Approach?

Different models capture different forms of information. ARIMA provides a classical statistical baseline, Prophet captures trend/seasonality with external regressors, gradient boosting handles nonlinear tabular relationships, and LSTM is designed for sequential patterns. Comparing the models also makes the project easier to audit and explain.

2. Objectives and Scope

2.1 Objectives

- Assemble a silver market dataset using daily prices, futures information, macroeconomic indicators, supply/demand and sentiment sources supplied for the project.
- Engineer technical indicators, rolling statistics, return lags, macro-financial ratios, positioning features and regime labels.
- Build and compare ARIMA, Prophet, XGBoost and LSTM forecasts.
- Generate simple, weighted, stacked and regime-switching ensemble outputs.
- Evaluate the models with RMSE, MAE, MAPE, directional accuracy and R2.
- Provide volatility, drawdown, VaR/CVaR and regime analytics for portfolio/risk interpretation.
- Create a Streamlit trading arena and a presentation-ready analytics dashboard.

2.2 Scope Boundaries

- The Streamlit trading arena is a historical simulation and does not execute real trades.
- The dashboard is an analytical reporting layer; it does not retrain models interactively.
- The implemented run_modeling.py pipeline is the source of the generated JSON and processed analytical outputs.
- The final merged analytical dataset begins in 2009 after cross-source alignment and rolling-feature warm-up, even though the daily source file is labelled 2000-2025.

2.3 Deliverable Mapping

Requirement	Implementation	Status
Data pipeline	Five project datasets loaded, aligned and merged	Implemented
ARIMA / seasonal baseline	Classical ARIMA baseline used for test forecasting	Implemented
Prophet	Prophet with macro regressors	Implemented
LSTM	Multivariate sequence model with 60-day lookback	Implemented
XGBoost	Gradient-boosted tabular regression	Implemented
Ensemble	Simple, weighted, stacked and regime-switching outputs	Implemented
Analytics	Risk, volatility, regime and market indicators	Implemented
Dashboard	Interactive analytics dashboard + PDF export	Implemented
Trading Arena	Historical signal-driven portfolio simulation	Implemented
Documentation	Project report, README and GitHub packaging	This report / remaining packaging

3. Data Sources and Dataset Engineering

The project uses the following supplied source datasets:

Dataset	Role in pipeline
silver_daily_ohlc_2000_2025.csv	Daily OHLCV, VWAP, returns and basic market variables
silver_futures_contracts.csv	Open interest, futures volume, basis and carry variables
silver_macroeconomic_monthly.csv	Rates, CPI, DXY, oil, VIX, M2, employment and PMI indicators
silver_sentiment_weekly.csv	CFTC positioning, news/social sentiment and Google Trends-style signals
silver_supply_demand_annual.csv	Annual supply-demand fundamentals

3.1 Alignment and Preprocessing

- Daily price records were sorted chronologically and indexed for time-series processing.
- Monthly macroeconomic variables were merged backward onto daily observations.
- Weekly sentiment / positioning variables were aligned backward to daily observations.
- Futures contracts were aggregated by trade date before merging.
- Rolling indicators introduced warm-up missing values; rows without the required long-window features were removed before modelling.
- The final merged analytical dataset contains 4,435 rows and 101 columns.

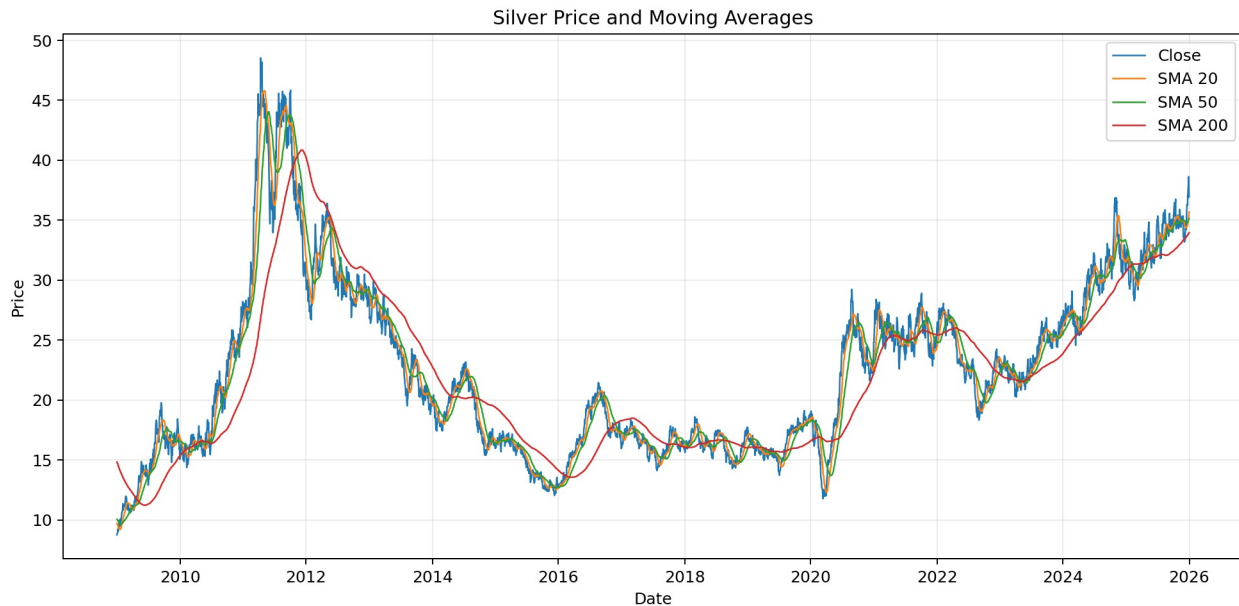


Figure 1. Silver close price with 20-, 50- and 200-day moving averages in the merged analytical dataset.

4. System Architecture

The project is organised as a data-to-decision pipeline. The Python modelling layer generates reusable JSON and CSV outputs; the presentation layer consumes those outputs without retraining.

System Architecture

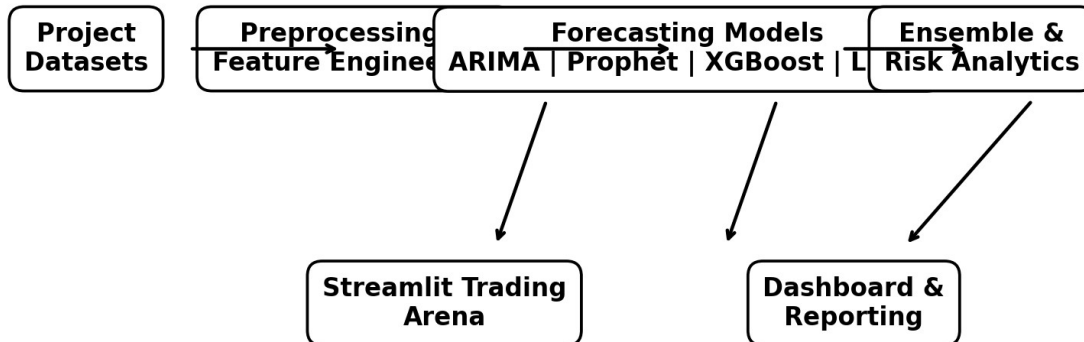


Figure 2. High-level system architecture.

4.1 Execution Flow

- Input layer: project-provided CSV datasets.
- Processing layer: chronological sorting, alignment, rolling calculations, macro/sentiment/futures merges and derived variables.
- Modelling layer: ARIMA baseline, Prophet, XGBoost and LSTM.
- Ensemble layer: simple averaging, inverse-RMSE weighting, stacking and regime-switching.
- Risk layer: volatility, drawdown, VaR/CVaR and regime classification.
- Application layer: Streamlit trading arena and analytics dashboard/PDF.

4.2 Project Outputs

- processed/merged_dataset.csv - engineered analytical dataset.
- web/model_data.json / trading_arena/model_data.json - test predictions, metrics, regimes and crisis scenario data.
- models/lstm_model.h5 and models/xgboost_model.pkl - saved model artefacts produced by the final run.
- dashboard/silver_commodity_dashboard.pdf - presentation-ready analytics dashboard.
- trading_arena/app.py - interactive Streamlit demonstration.

5. Feature Engineering

The pipeline converts raw market and external variables into a broad set of technical, volatility, macro-financial and positioning features.

5.1 Technical Indicators

- Simple moving averages: SMA 20, SMA 50 and SMA 200.
- Exponential moving averages: EMA 12 and EMA 26.
- MACD, MACD signal and histogram.
- RSI(14).
- Bollinger Bands and band width.
- ATR(14), stochastic oscillator and simplified ADX.
- OBV and volume ratio.
- Ichimoku conversion/base components and rolling Fibonacci reference levels.

5.2 Returns and Volatility

- Daily percentage/log returns.
- Lagged returns at 1, 2, 3, 5, 10, 20 and 60 periods.
- Parkinson volatility.
- 20-, 60- and 252-day rolling annualised close-return volatility.

5.3 Macro, Sentiment and Futures Features

- Real interest rate and yield-curve slope.
- Gold/silver, oil/silver and macro ratio variables.
- DXY change, M2 growth and selected macro regime indicators.
- CFTC speculative and commercial net positioning.
- COT index and Google Trends momentum.
- Futures open interest, aggregate volume, basis, basis percentage and implied carry.

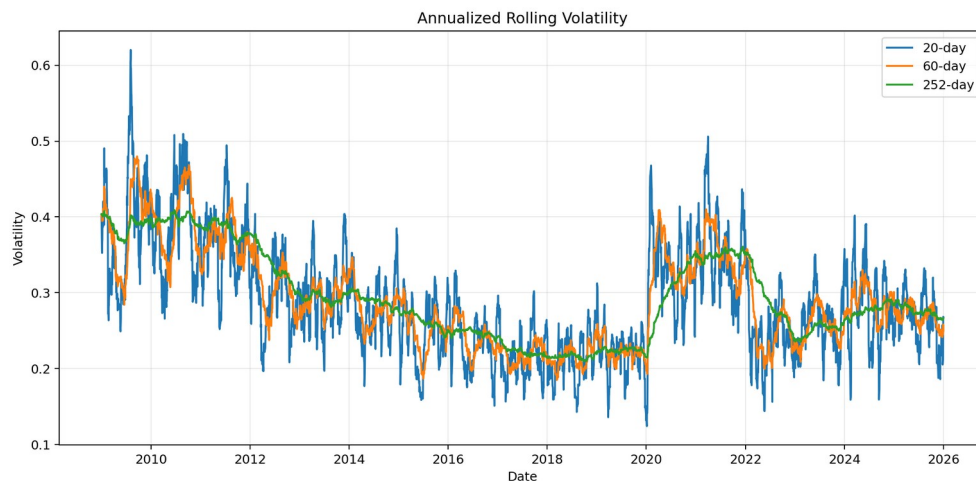


Figure 3. Rolling annualised volatility features used for risk monitoring.

6. Forecasting Methodology

6.1 Train / Validation / Test Design

The final modelling pipeline uses a chronological split to reduce leakage risk:

Split	Period	Purpose
Training	Before 2018-01-01	Model fitting
Validation	2018-01-01 to 2020-12-31	Intermediate model assessment / weighting context
Test	2021-01-01 to 2025-12-31	Out-of-sample reporting

6.2 ARIMA Baseline

A classical ARIMA-style baseline is included to provide a statistical benchmark. In the final `run_modeling.py` path, the daily test forecast is generated with a Ridge-on-lag proxy labelled as the ARIMA baseline for computational practicality, while the Auto-ARIMA dependency is optional. This implementation choice should be considered when interpreting the baseline results.

6.3 Prophet with Macroeconomic Regressors

Prophet is configured with yearly and weekly seasonality and external regressors including gold price, DXY, VIX and real interest rate. A linear-regression fallback is used when Prophet is unavailable or fails.

6.4 XGBoost

XGBoost is trained on engineered technical, macro, positioning and lag features. The productionized run uses 300 trees, max depth 5, learning rate 0.03, subsampling and column subsampling. The target is next-day close.

6.5 LSTM

The neural model uses a 60-day sequence lookahead and a two-layer LSTM architecture followed by dense layers. The pipeline uses MinMax scaling and a 10-epoch training run. A scikit-learn MLP fallback is included if TensorFlow execution fails.

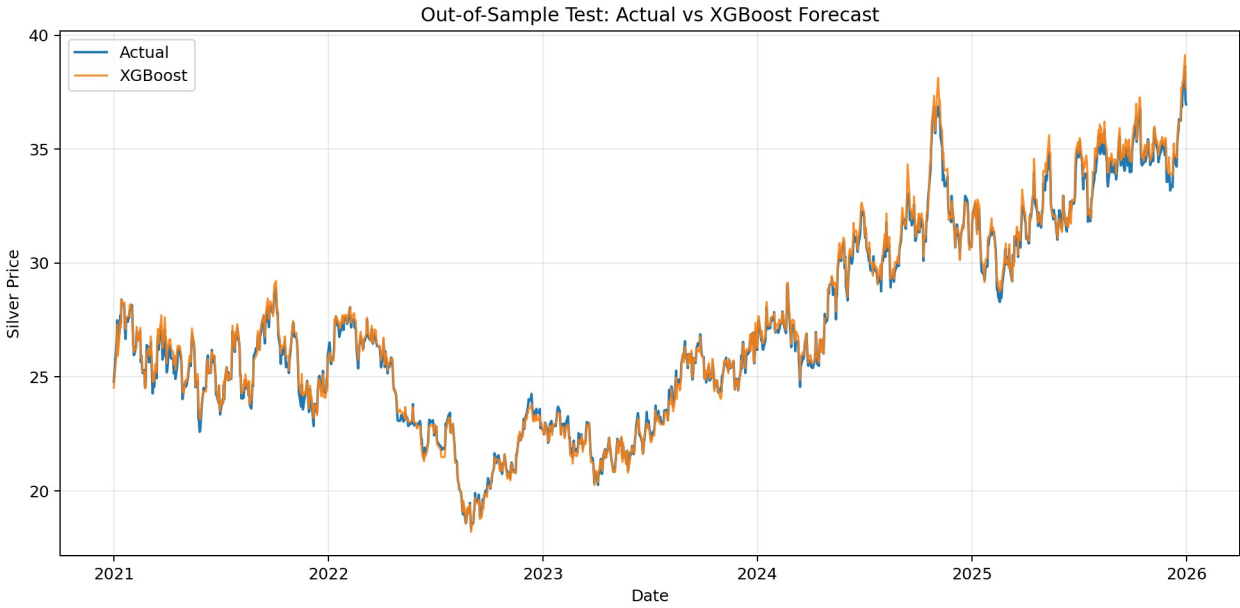


Figure 4. Out-of-sample actual silver price versus the XGBoost forecast on the 2021-2025 test period.

7. Ensemble and Risk Analytics

7.1 Ensemble Strategies

- Simple average of ARIMA, Prophet, XGBoost and LSTM predictions.
- Inverse-RMSE weighted ensemble using validation error weights.
- Ridge stacking meta-learner over the four model outputs.
- Macro-regime switching that changes model weights for crisis, expansionary and contractionary states.

Important implementation note: the stacking ensemble in `run_modeling.py` uses synthetic/mock validation predictions and the weighted ensemble uses hard-coded illustrative validation RMSE values. These outputs are therefore best interpreted as demonstration ensembles rather than a rigorously optimised research ensemble.

7.2 Risk Metrics

- Rolling volatility at 20, 60 and 252-day windows.
- Maximum drawdown of the test-period return series.
- Historical 95% and 99% daily VaR.
- Historical 95% CVaR.
- Volatility-based regime mapping with optional Hidden Markov Model support.
- Macro regime classification using VIX and PMI-style conditions.

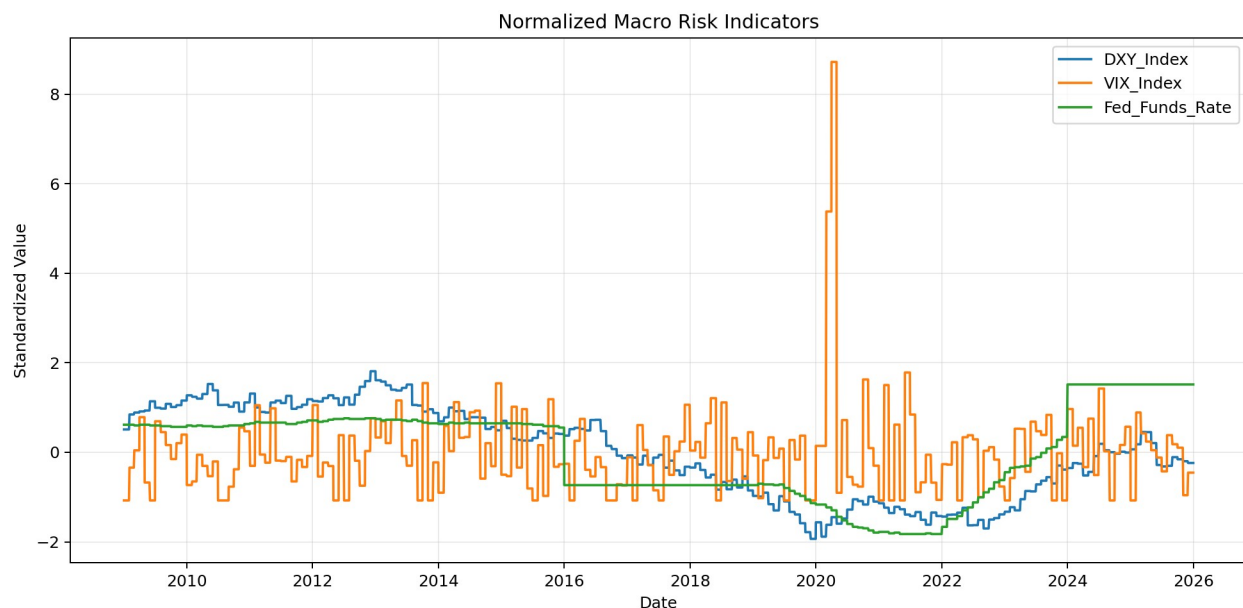


Figure 5. Normalised DXY, VIX and Federal Funds Rate signals for market-risk context.

8. Evaluation and Results

The reported metrics are computed on the 1,304-observation out-of-sample test period from 2021-01-01 to 2025-12-31.

Model	RMSE	MAE	MAPE	Directional Accuracy	R2
ARIMA	7.0542	5.7007	19.2194%	51.65%	-1.5763
Prophet	20.0263	18.8433	67.6256%	47.35%	-19.7640
XGBoost	0.2680	0.2046	0.7345%	85.11%	0.9963
LSTM	2.2669	2.0518	7.8281%	48.20%	0.7339
Ensemble_Simple	6.2099	5.5586	19.4674%	79.59%	-0.9966
Ensemble_Weighted	5.0318	4.4614	15.5700%	80.51%	-0.3109
Ensemble_Stacked	3.0433	2.7217	9.5506%	83.19%	0.5205
Ensemble_Switching	5.9682	4.6131	15.9801%	75.83%	-0.8441

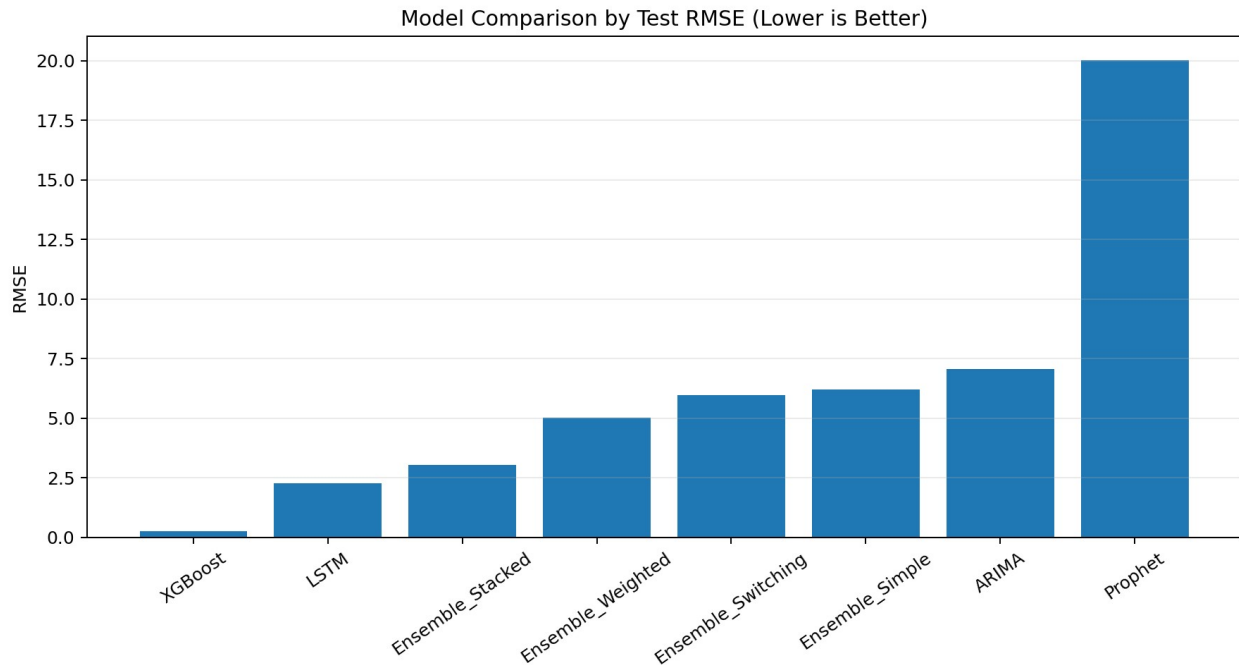


Figure 6. Test-period RMSE comparison across individual models and ensembles.

8.1 Interpretation

XGBoost is the best-performing model in the exported test results by a wide margin. Its RMSE of 0.2680 and R2 of 0.9963 are substantially better than the other reported models. It also records 85.11% directional accuracy. The stronger result is consistent with the highly engineered tabular feature set available to XGBoost.

The ensemble models improve directional performance in several cases, but they do not beat XGBoost on RMSE in this exported run. The simple and regime-switching ensembles retain negative R2, while the stacked ensemble reaches R2 0.5205. The result shows why ensemble

methods should be validated independently rather than assumed to outperform the strongest base learner.

The negative R2 and higher error values of the ARIMA and Prophet baselines indicate that the classical/seasonal baselines were not competitive with the feature-rich tree model on this specific test period.

9. Dashboard and Trading Arena

9.1 Analytics Dashboard

The project includes a presentation-ready dashboard built from the processed analytical dataset. It contains an overview of price and moving averages, macroeconomic/risk monitoring, sentiment and positioning indicators, and portfolio/trading-oriented views.

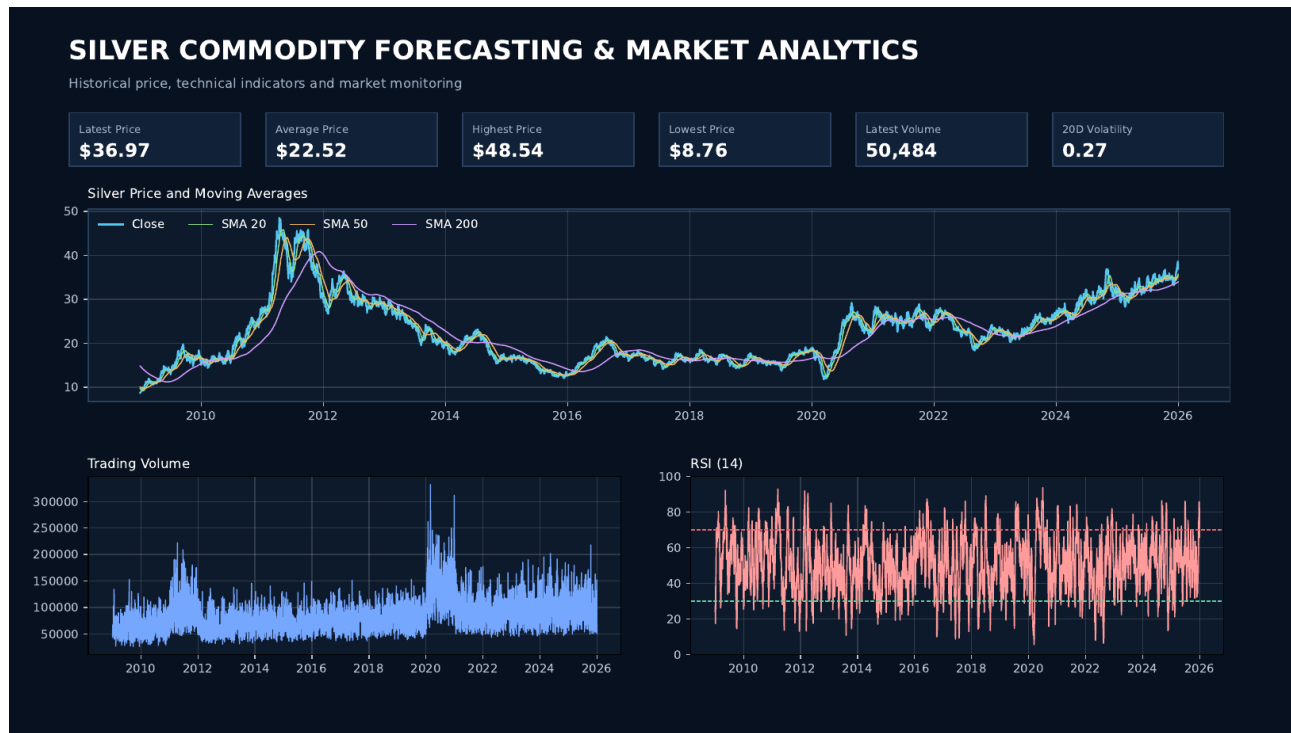


Figure 7. Dashboard - Silver commodity forecasting and market analytics overview.

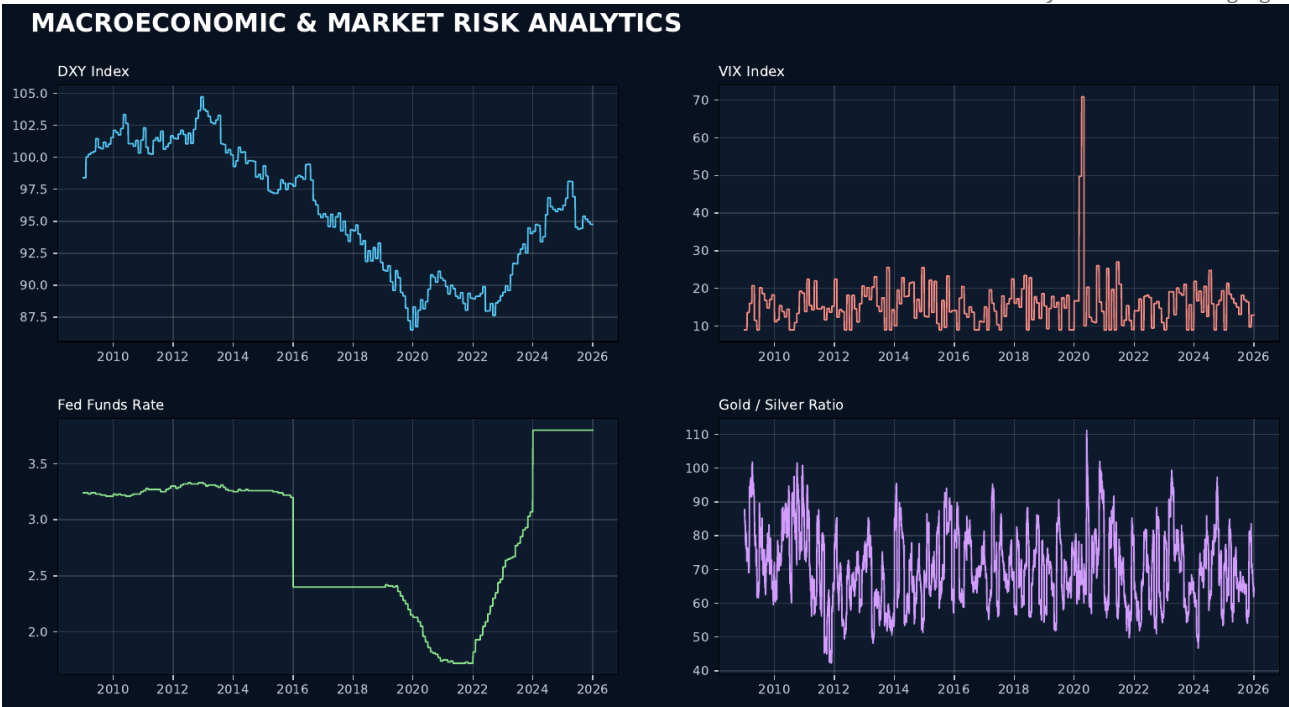


Figure 8. Dashboard - Macroeconomic and market-risk analytics.

9.2 Streamlit Trading Arena

The Streamlit application reads the exported `model_data.json` file and provides a model selector, forecast charts, metrics, scenario exploration and a historical signal-based trading simulator. The simulator uses the selected model forecast versus the realised test-period price to create BUY/SELL/HOLD actions under a configurable threshold and tracks cash, holdings, portfolio value, P/L and trade history.

9.3 User Flow

- Select a model from the sidebar.
- Review the current test-period price, forecast and signal.
- Inspect actual versus predicted price charts.
- Review model metrics and risk indicators.
- Run the historical trading simulation with a chosen starting capital and signal threshold.
- Explore predefined historical crisis scenario paths.

9.4 Practical Interpretation

The dashboard and arena are decision-support demonstrations. They do not constitute financial advice, live trading infrastructure or a guarantee of future performance.

10. Limitations and Responsible Interpretation

- The final merged analytical dataset spans 2009-2025 after alignment and rolling-feature warm-up, although the daily source file is named 2000-2025.
- The exported modelling pipeline uses a Ridge lag proxy for the ARIMA baseline in the final daily forecast path.
- The ensemble weighting / stacking implementation contains illustrative validation values and synthetic validation predictions; these should not be presented as a rigorously tuned ensemble experiment.
- No live market feed is connected. The Streamlit arena and dashboard are based on exported historical/test results.
- The project uses simulated trading and should not be used to make real investment decisions.
- Forecasting performance can vary materially across market regimes and future periods.

10.1 Data and Modelling Quality Considerations

Future iterations should implement a fully walk-forward validation scheme for every model, retrain the ensemble weights using genuine validation forecasts, add a true seasonal ARIMA/SARIMA benchmark if required by the brief, and compare results across multiple rolling test windows.

11. Conclusion and Future Scope

The completed project demonstrates a practical end-to-end commodity forecasting system that integrates multi-source market data, technical and macro feature engineering, multiple forecasting paradigms, ensemble outputs, risk analytics, scenario analysis and interactive visualisation. The exported test results show XGBoost as the strongest model in this run, while the Streamlit and dashboard layers convert the analytical outputs into an accessible decision-support prototype.

11.1 Future Scope

- Implement genuine rolling walk-forward retraining and hyperparameter tuning.
- Replace illustrative ensemble weights with validation-derived weights and out-of-fold stacking forecasts.
- Add SARIMA as a fully evaluated seasonal baseline.
- Introduce probabilistic forecasts and prediction intervals.
- Add SHAP-based explainability for XGBoost.
- Connect the application to a live/periodic market data service if permitted by project requirements.
- Add automated model monitoring for drift and forecast degradation.
- Expand the trading arena into multiple portfolio/risk scenarios and transaction-cost assumptions.

Appendix A - Project Structure and Reproduction

Recommended final submission structure:

```
Commodity-Price-Forecasting-Agent/
├── dashboard/
│   └── silver_commodity_dashboard.pdf
├── data/
├── models/
│   ├── lstm_model.h5
│   └── xgboost_model.pkl
├── processed/
│   └── merged_dataset.csv
├── report/
│   └── Commodity_Price_Forecasting_Agent_Report.pdf
├── trading_arena/
│   ├── app.py
│   └── model_data.json
├── run_modeling.py
├── requirements.txt
└── README.md
```

A.1 Run the Forecasting Pipeline

In the project environment, install the Python requirements and run `run_modeling.py` after updating `DATA_DIR` / output paths for the target environment. The completed pipeline exports the processed dataset, model artefacts and `model_data.json`.

A.2 Run the Streamlit Arena

Open a terminal in the `trading_arena` folder and run:

```
streamlit run app.py
```

A.3 Main Generated Artifacts

Artifact	Purpose
<code>merged_dataset.csv</code>	Processed analytical dataset for dashboarding
<code>model_data.json</code>	Test predictions, metrics, regimes and scenario data
<code>lstm_model.h5</code>	Saved neural sequence model artefact
<code>xgboost_model.pkl</code>	Saved gradient-boosting model artefact
<code>silver_commodity_dashboard.pdf</code>	Static dashboard/reporting output
<code>trading_arena/app.py</code>	Interactive Streamlit user interface

Appendix B - Key Test Results

Key result snapshot from the exported test-period metrics:

Measure	Value
Best model	XGBoost
Best RMSE	0.2680

Best MAE	0.2046
Best MAPE	0.7345%
Best Directional Accuracy	85.11%
Best R2	0.9963
Test observations	1,304
Test period	2021-01-01 to 2025-12-31